TUGAS

Algoritma dan Struktur Data

Nama : Rachmah Nur Chotimah

NIM : 254107060052

Kelas : SIB 1A

No.Absen : 20

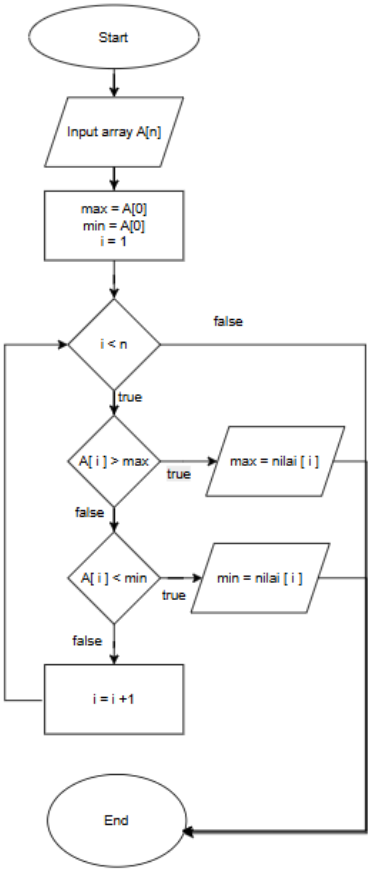
1. Buatlah flowchart untuk menentukan nilai maksimal dan minimal dari sekumpulan nilai dengan algoritma Brute Force dan Divide Conquer!
2. Buatlah flowchart untuk menghitung hasil penghitungan deret fibonacci dengan algoritma Brute Force dan Divide Conquer!
3. Tentukan notasi Big O yang sesuai dari kode program berikut!

```javascript
function search(array, value) {
  for (let i = 0; i < array.length; i++) {
    if (array[i] === value) {
      return i;
    }
  }
  return -1;
}
let score = [12, 22, 45, 67, 96];
console.log(search(score, 100));
```

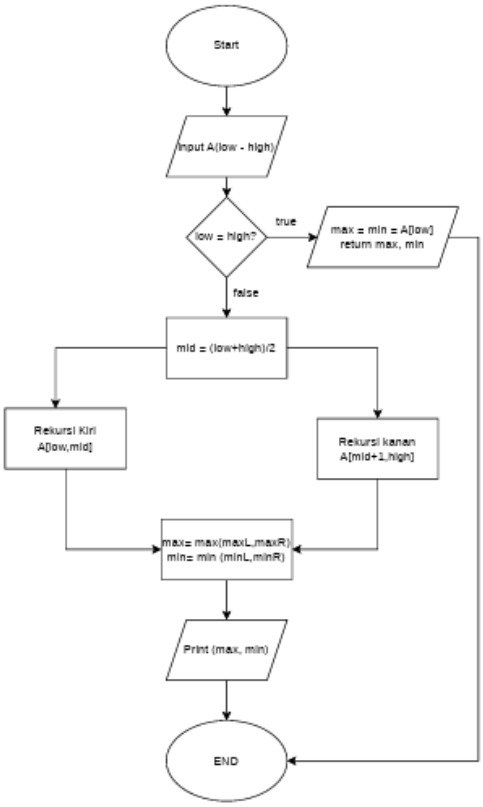
```javascript
function binarySearch(array, value) {
  let firstIndex = 0;
  let lastIndex = array.length - 1;
  while (firstIndex <= lastIndex) {
    let middleIndex = Math.floor((firstIndex + lastIndex) / 2);
    if (array[middleIndex] === value) {
      return middleIndex;
    }
    if (array[middleIndex] > value) {
      lastIndex = middleIndex - 1;
    } else {
      firstIndex = middleIndex + 1;
    }
  }
  return -1;
}

let score = [1, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59];
console.log(binarySearch(score, 37));
```

Jawab :

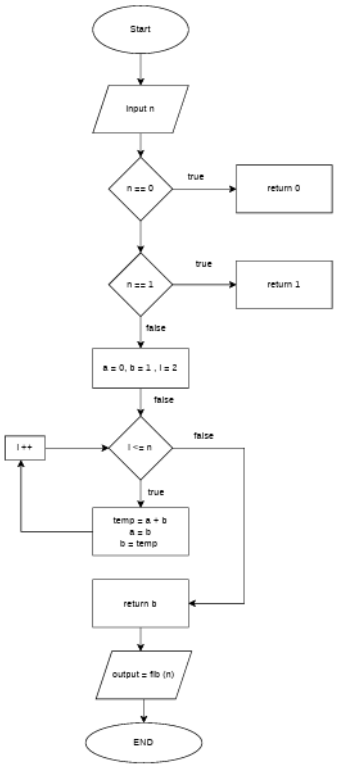
1. a.Brute Force

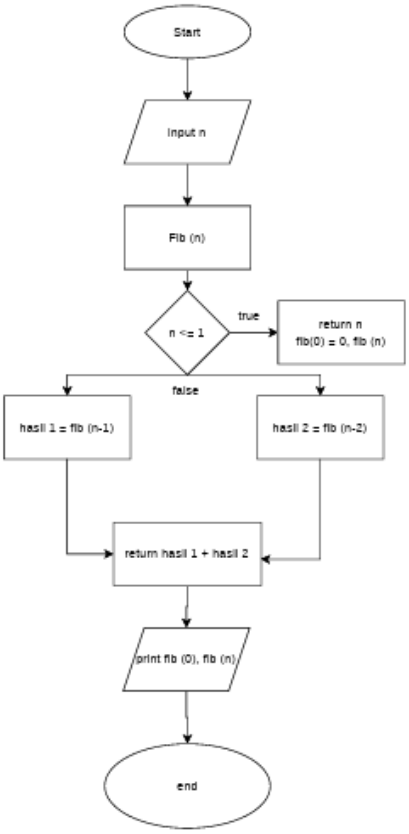


b. Divide dan Conquer



2. a.Brute Force b.Divide dan Conquer





3. Berdasarkan aturan Big O, kita mencari bagian yang paling berkontribusi pada kompleksitas waktu:

a. Linear Search

```
function search (array, value) {
for (let i=0; i < array.length; i++) { // Looping bergantung panjang n
 if (array[i] === value) { return i; }
 }
  return -1;
}
```

- **Notasi: O(n)**.
- **Alasan:** Algoritma menggunakan perulangan tunggal yang jumlah iterasinya berbanding lurus dengan panjang array (n).

b. Binary Search

```
function binarySearch(array, value) {
    while (firstIndex < lastIndex) {
        let middleIndex = Math.floor((firstIndex + lastIndex) / 2); // Membagi
    data jadi dua
// ... logika eliminasi setengah data ...

 }
```

}

- **Notasi: O(log n)**.
- **Alasan:** Algoritma ini membagi ruang pencarian menjadi dua bagian pada setiap iterasinya, sehingga jumlah langkah yang diperlukan adalah logaritmik terhadap ukuran input (n).